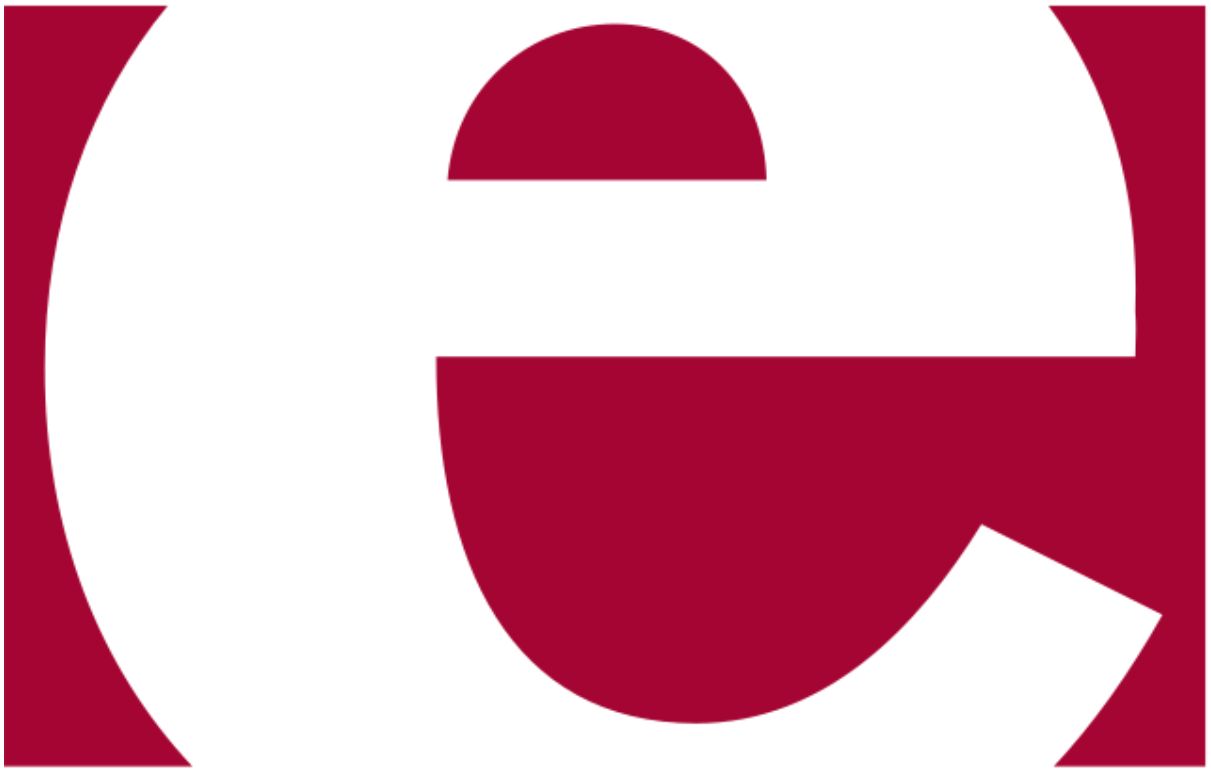
Mini Version Control System

PCLT Final Project



ERLANG

Authors:

Daniel Eugénio 59797

Renato Viola 60665

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

In this report, we discuss the implementation of a simulation of a small Version Control System. For every operation, the user performing it gets a response from the server in the form of a message, whether it be indicating that the operation was successful or that an error occurred. Each error case has its own specific message identifying it.

Interaction Protocol

General-purpose commands

Register

Registers a new user on the system. Verification checks if a user with that name is already signed up. Success message is returned if the username isn't already taken. When the client signs up, a new process is created to listen to notifications. Each client has a "companion" process for this.

New

Creates a new repository in the system. When a user creates a new repository in the system, they are automatically registered as a subscriber of that repository, so their listener process is added to the subscriber list, so the owner is always notified of changes.
Error cases are the following:

- There is no user by the given name;
- There is no repository by the given name;

Invite

A user that owns any repository can invite another user to be a collaborator in that specific repository. The second user is added to the list of collaborators of that repo and can then access and modify it.
Error cases are the following:

- There is no repository by the given name;
- The given administrator is not a registered user;
- The given administrator is not the actual administrator;
- The invited user is not registered in the system;
- The invited user is the administrator himself;
- The invited user is already a collaborator.

Subscribe

A user that is a collaborator in someone else's repository can subscribe to be notified of any changes in the repository files. To achieve this, their listener process is added to the subscriber list, so that the user can be informed of any changes.
Error cases are the following:

- There is no user by the given name;
- There is no repository by the given name;
- The user is not a collaborator in the given repository;
- The user is already subscribed for notifications in the given repository.

Repository specific commands

Add

Adds a new file to the specified repository, notifies the repository's subscribers.
Error cases are the following:

- There is no user by the given name;
- There is no repository by the given name;
- Given user is neither an admin nor a collaborator to the given repository;
- The file already exists in the repository.

Push

Updates a list of files in the specified repository, notifies the repository's subscribers.
Error cases are the following:

- There is no user by the given name;
- There is no repository by the given name;
- Given user is neither an admin nor a collaborator to the given repository;
- At least one of the provided filenames does not exist in the given repository.

Status

Returns a structure containing the repository's details, such as the admin, collaborators and a list of the files present in it.
Error cases are the following:

- There is no user by the given name;
- There is no repository by the given name;
- Given user is neither an admin nor a collaborator to the given repository;

Token

Adds a user to a repository's token list. A user can either successfully complete other operations like add or push if they're a collaborator or are present in this token list with a valid token.
Error cases are the following:

- There is no repository by the given name;
- There is no user (admin) by the given name;
- The given admin doesn't match the repository owner;
- The invited user doesn't exist;
- The invited user is someone who already has access to the repository;
- The token's validity is invalid

Solution explanation and properties

The server itself is implemented following the 'gen_server' behaviour pattern, which abstracts a good part of the typical client-server architecture boilerplate code we would have to write otherwise, decreasing the opportunities for error on our part.

Not only that, it allows us to manage all the user and repository information in a thread safe organized environment, since even simultaneous requests to the server are processed one at a time and in the order in which they are received.

In this sense, simultaneous operations won't interfere with each other. Handling an erroneous request isn't an issue either, as the server won't crash and it's just a matter of sending the adequate error message to the unfulfilled client.

When it comes to the operations 'add' and 'push' which notify all the subscribers of a repository, we made the dispatching of the notifications themselves an asynchronous task, so that the server can reply to the client who made the operation without blocking. As for the 'token' functionality, we opted for storing repository access tokens in a map inside the *repo* record. A token can be identified by the username of the invited user and its value is the expiration time of when the user loses access to the repository. To aid with the validation of tokens, an auxiliary function was created that verifies if a token belongs to a given user and if it is still valid.

We introduced a lightweight client that simply makes calls to the server and prints out the responses in an easy-to-read format. The client is also responsible for the creation of notification listeners that run "on the side" of each client.

A test module was also developed to better assess the correctness of the functionalities the server provides. It executes all the different commands a user has access to and simulates interactions between two concurrent clients and a single server, as well as the collaboration between users.